Módulo 5: Liderazgo técnico

> **Contexto.** Equipo de 8 desarrolladores, 40 % de deuda técnica en servicios clave, 10 incidentes críticos el último mes, una funcionalidad comprometida para dentro de 3 semanas, sin estándares, *code review* irregular y dos juniors con brechas técnicas importantes.

Respuesta corta. Estabilizaría antes de acelerar. En las dos primeras semanas me concentraría en contener los incidentes, acordar con negocio un MVP que sí quepa en 3 semanas e instalar unos pocos estándares obligatorios (DoD, revisión y CI con *quality gate*). Organizaría al equipo en dos frentes con rotación y mediría el avance con métricas DORA a nivel de equipo.

1. Cinco prioridades para las primeras 2 semanas

1. **Contener los incidentes:** guardia rotativa, clasificación por severidad y un análisis de los 10 incidentes para encontrar las 2–3 causas que más se repiten.
2. **Acordar el alcance de la funcionalidad:** recortarla a un MVP con negocio, con los riesgos escritos.
3. **Diagnosticar la deuda con datos:** cruzar deuda, incidentes y frecuencia de cambio por servicio, y atacar primero donde coinciden.
4. **Instalar estándares mínimos:** DoD, revisión obligatoria con checklist y CI con *quality gate* en lo que se toque.
5. **Conocer a las personas:** 1:1 con los 8, y un plan de crecimiento con *pairing* para los dos juniors.

Días (hábiles)	Foco	Resultado al cierre del bloque
1–2	Escuchar y medir: 1:1, revisión de los incidentes, del pipeline y de cómo se despliega hoy	Línea base: incidentes por servicio y causa, métricas DORA aproximadas, mapa de riesgos
3–5	Contener y acordar: guardia rotativa, *runbooks* y alertas para las causas más frecuentes; sesión con negocio para definir el MVP	Guardia operando; MVP y riesgos acordados por escrito
6–8	Estándares: DoD, plantilla de PR con checklist, rama `main` protegida, *quality gate* en CI aplicado al código nuevo ("*clean as you code*")	Reglas activas desde ese momento; nada se fusiona sin revisión y CI en verde
9–10	Organizar y comunicar: frentes de trabajo, parejas, backlog de deuda priorizado, primera retrospectiva, informe a negocio con datos	Plan desde la semana 3 y expectativas alineadas

2. Organización del equipo

Frente	Personas	Responsabilidad
Entrega	5 (3 seniors o semi-seniors + 1 junior + 1 apoyo de QA/DevOps si existe)	MVP de la funcionalidad, detrás de *feature flag*
Estabilización	3 (1 senior + 1 semi-senior + 1 junior)	Incidentes, causas raíz y deuda de los servicios clave

- **Rotación cada sprint (2 semanas):** una o dos personas cambian de frente. Así el conocimiento se comparte y estabilizar no se vuelve "trabajo de segunda".
- **Guardia de incidentes semanal:** una persona primaria (del frente de estabilización) y un senior de respaldo. Protege la concentración del frente de entrega.
- **Juniors:**
  - *Pairing* diario con un senior, no solo revisión al final, y tareas acotadas con criterios de aceptación claros.
  - Plan de 30-60-90 días sobre sus brechas concretas (por ejemplo, pruebas, SQL o seguridad).
  - Revisiones de código con retroalimentación que explique el *por qué*.
  - Entran a la guardia acompañados después del primer mes.
- **Cadencia:** daily de 15 min, refinamiento semanal con negocio, demo y retrospectiva cada 2 semanas.

3. Métricas del área

Son **métricas de equipo para mejorar el sistema, no para evaluar personas**. Usadas individualmente se distorsionan (se optimiza el número, no el resultado). Primero se mide la línea base y después se fijan metas de tendencia.

Métrica	Qué indica	Meta inicial (a validar con la línea base)
Frecuencia de despliegue (DORA)	Capacidad de entregar en lotes pequeños	Al menos semanal por servicio
Lead time de cambios (DORA)	Tiempo de commit a producción	Reducción sostenida mes a mes
Tasa de fallos en cambios (DORA)	Calidad de lo que se despliega	< 15 %
MTTR (DORA)	Capacidad de recuperación	< 1 h para severidad 1
Incidentes críticos por mes y por servicio	Estabilidad (hoy: 10)	Tendencia a la baja; p. ej. la mitad en un trimestre
Quality gate de SonarQube (código nuevo)	Que no se agregue deuda	100 % de los PR pasan: 0 bugs o vulnerabilidades críticas, cobertura ≥ 80 %
Ratio de deuda técnica en servicios clave	Evolución del 40 %	Baja trimestre a trimestre
Predictibilidad del sprint	% de compromisos cumplidos	≥ 80 %
Tiempo hasta la primera revisión de un PR	Salud del *code review*	< 1 día hábil

4. Prácticas técnicas obligatorias

Práctica	Regla concreta
Definition of Done	Código revisado, pruebas automatizadas, *quality gate* en verde, documentación o ADR si aplica, desplegado en *staging* y con observabilidad (logs y métricas)
Code review	Al menos 1 aprobación (2 en servicios críticos), PR de menos de ~400 líneas. Checklist: funcionalidad, pruebas, seguridad OWASP (validación de entrada, autorización, secretos, sin datos personales en logs), manejo de errores y observabilidad
CI con *quality gate*	Build, pruebas, cobertura del código nuevo, análisis estático (SAST) y de dependencias. `main` protegida: no se fusiona con CI en rojo
Ramas y commits	Ramas cortas (`feature/*` , `fix/*`) sobre `main` , *Conventional Commits*, PR enlazado a la historia de usuario
Postmortems sin culpa	Para severidad 1 y 2, en menos de 5 días; las acciones tienen dueño y fecha y se siguen en el backlog
ADRs	Toda decisión con impacto (tecnología, contrato o patrón) se registra con contexto, decisión y alternativa descartada
Uso responsable de IA	Se permiten asistentes de código, pero el código generado pasa **la misma revisión y las mismas pruebas**, el autor responde por él, no se incluyen datos sensibles ni secretos en los *prompts* y solo se usan herramientas aprobadas por la empresa

5. Gestionar la presión del negocio sin comprometer la calidad

- **El alcance se negocia, la calidad no.** La fecha se cumple recortando a un **MVP** (priorización MoSCoW con negocio), no saltándose el DoD.
- **Riesgos con datos:** 10 incidentes críticos al mes tienen un costo para el negocio. Presento ese costo junto con lo que implicaría entregar sin pruebas, para que la decisión de alcance sea conjunta e informada.
- ***Feature flags*:** desplegar temprano y oculto, activar gradualmente y apagar sin redesplegar si algo falla.
- **Transparencia semanal:** un semáforo de alcance, fecha y riesgos, avisando temprano y no el día anterior.
- **Capacidad fija para deuda:** ~20 % de cada sprint, que no se renegocia sprint a sprint, ligada a la reducción de incidentes medida en §3.

6. Evidencia de estas prácticas en esta prueba

Desarrollé esta prueba con un asistente de IA y revisé y verifiqué cada entregable. Varias de las prácticas que propongo las apliqué aquí, a escala de una persona:

Práctica propuesta	Cómo la apliqué en la prueba
CI con *quality gate*	JaCoCo hace fallar el build del backend si la cobertura de líneas baja del 80 % (96 pruebas)
ADRs	`DECISIONES.md` en backend y frontend: cada decisión con su alternativa descartada y los supuestos tomados
Pruebas más allá de las unitarias	Las pruebas e2e con Playwright encontraron **3 defectos reales** que las unitarias no detectaban (recarga tras un 409, un 403 de CORS detrás de nginx y la zona horaria del contenedor)
IA con revisión humana	Todo lo generado se verificó con build, pruebas, revisiones visuales en el navegador y mediciones reales (por ejemplo, los planes de ejecución del Módulo 3)
Honestidad sobre el estado	El diseño (Módulo 1) distingue en una tabla lo implementado, lo simplificado y lo propuesto